PeriodicWave（wada_dev）のファイルの地図 —— λ-jellium の WC と AHC の計算

どの工程でどのファイルが呼ばれるか。①〜⑦ は工程の順で、⑤ は ④ の中で毎回呼ばれる。2026-11-02 の wada_dev による。

箱の枠の色 = 走る場所：PC（Windows）／計算サーバ（Slurm・GPU）／クラウドか PC の CPU／ネットワークの中（⑤）

矢印：呼ぶ（実行・import）／ファイルを渡す　　箱：コード／データのファイル

① 出発点の HF の表を作る
クラウドか PC の CPU

screening_analysis/lj_hf_branch.py
λ ごとに WC 型と AHC 型の HF を解く。λ に沿って前の λ の解を種にし、チャーン数 C が変わらなければ採る

import

periodicwave/pbc/lj_hf.py
クラス LJHF：下のバンドに射影した HF を 6×6 の $\boldsymbol{k}$ で解く（solve）、チャーン数（chern）、表を書く（orbitals_text）

書く

hf_N36_lam{λ}_{wc|ahc}.txt
HF の Bloch 軌道の係数の表（λ 6 点 × 2 型 = 12 枚）。PC の **ws_pw_kit/lj2_hf/** に置く

② 投入
PC（学内か VPN）から計算サーバへ

ws_pw_kit/_log_lj3.bat
ダブルクリックで実行。条件を環境変数で決める：λ、型、タグ、PW_INPUT12=1（入力 12 成分）、PW_TRIM_FIX=1（Trim の幅を固定）、690 分で止める

実行（λ ごとに 1 回）

ws_pw_kit/ws_go_lj3.ps1
(1) コード 8 本・**pw_lj3.sh**・HF の表を計算サーバへ送り、md5 で確かめる
(2) 計算サーバで import を確かめる
(3) sbatch　(4) 待ち行列を表示
ssh の設定は **ws_config.ps1**

sbatch

jobs/pw_lj3.sh（**ws_pw_kit/pw_lj3.sh** を送ったもの）
1 ジョブで WC 型と AHC 型の 2 本（GPU 1 枚に 1 本）。各本で
`python periodicwave/configs/run_lambda_jellium.py N r_s λ 反復数 タグ`
HF の表は環境変数 PW_LJ_HF で渡す

③ 設定
計算サーバ

periodicwave/configs/run_lambda_jellium.py
設定 cfg を作る（既定値は **base_config.py**）
・系：6×6 の三角格子のセル（**pbc/lattices.py**）、局所エネルギーは pbc.lambda_jellium.local_energy、擬スピンあり
・ネットワーク：CustomScreenformer（1 層 16 ヘッド、窓 smooth7 を固定、入力 12 成分の高調波 crystal_harmonics、軌道は HF の表）
・出発点：HF の密度の山の格子にウォーカーを置く
・保存先（run ごと）：
results/lambda-jellium/CustomScreenformer/N36_rs15_lam{λ}_{タグ}/

train_screen.train(cfg) を呼ぶ（④）

④ 学習のループ
計算サーバの GPU

periodicwave/train_screen.py
初期化：ネットワークを作る（CustomScreenformer.make_fermi_net、⑤）。ウォーカーは格子点のまわり、擬スピンは乱数

1 step（約 4 万回くりかえす）
1. サンプル：**mcmc.py**。全電子を動かす提案 1 回ごとに、擬スピン反転の提案（mh_spin_flip）を 3 回
2. 局所エネルギー $E_L = H\psi/\psi$：**pbc/lambda_jellium.py**（運動エネルギーと λ の項、擬スピン反転の項、Δn_-）。クーロン（Ewald）は **pbc/hamiltonian.py**
3. 勾配：**loss.py**。$E = \langle E_L\rangle$、$\partial E/\partial\theta = 2\,\mathrm{Re}\langle (E_L - E)\,O^*\rangle$、$O = \partial \ln\psi/\partial\theta$
4. 更新：KFAC（kfac_jax、**curvature_tags_and_blocks.py**）

Δ：上のバンドを抑えるエネルギー（$2/r_s$ Ry）、n_-：擬スピン -1 の電子の数、θ：⑤ の学習で動くパラメータ

書く

書き出し：**checkpoint.py** → qmcjax_ckpt_*.npz（15 分ごと）、**utils/writers.py** → train_stats.csv。690 分で自分で止まる（PW_MAX_MINUTES）

⑥ 確認と回収
PC（学内か VPN）、**ws_pw_kit/**

_log_check_lj3.bat → **ws_check_lj3.ps1**
読むだけ：待ち行列、GPU、ログの頭と尻、train_stats → _logs/check_lj3.log

_log_get_lj3.bat → **ws_get_lj3.ps1**
最後のチェックポイント、config.json、train_stats.csv、ログを取ってくる（比べる lj2 の run も）

書く

回収した run
ws_pw_kit/pulled_lj3/results/lambda-jellium/CustomScreenformer/N36_rs15_lam{λ}_{タグ}/
lj2 の run は **ws_pw_kit/pulled_lj2/**

_log_lj3_stop.bat → **ws_stop_lj3.ps1**
lj3 のジョブを取り消す（チェックポイントは残る）

⑦ 解析
クラウドの CPU、**screening_analysis/lj3/**

lj3net.py
チェックポイントからネットワークを作り直す（**../sampler.py**、**CustomScreenformer.py**）

import

readout3.py
1 層目のアテンションの中身（s^h_{ij}、α、m_h）。ネットワークの出力と相対 5×10^{-7} で一致

import

analyze_all.py → ana_all.json
C3 固有値（**../lj2/c3hf.py**）、擬スピンの模様、擬スピンの組ごとの重み

energies.py（エネルギー）、**input12_check.py**（入力の対称性）、**transfer.py**・**swap.py**・**moved.py**（WC 型と AHC 型の差、配置の入れ替え、パラメータの動き）
lj2 の 12 本の解析は **screening_analysis/lj2/**

⑤ 1 回の $\psi_\theta(R)$ の計算　periodicwave/CustomScreenformer.py の make_fermi_net（④ の 1〜3 で毎回使う）

ψ と微分を計算（毎 step）

入力
電子 i の位置 $\boldsymbol{r}_i$ と擬スピン $s_i = \pm1$（$i = 1,\dots,36$）
$R = \{(\boldsymbol{r}_i, s_i)\}$：全電子の配置

特徴　**pbc/feature_layer.py**
$(\sin 2\pi\boldsymbol{u}_i,\ \cos 2\pi\boldsymbol{u}_i)$、$\boldsymbol{u}_i = A^{-1}\boldsymbol{r}_i$（$A$：セルの格子ベクトル）
対の距離 d_{ij}（周期ノルム）

1 電子の入力と埋め込み
one_electron_inputs
上の 4 成分 + 結晶の高調波 $\cos \boldsymbol{G}\cdot\boldsymbol{r}_i$、$\sin \boldsymbol{G}\cdot\boldsymbol{r}_i$（$\boldsymbol{G}$ 3 本、crystal_harmonics）+ one-hot (n_+, n_-) = 12 成分
埋め込み $\boldsymbol{x}_i = W_{\text{emb}}\times$（12 成分）

遮蔽アテンション　make_screening_attention
$\boldsymbol{q}_i, \boldsymbol{k}_j, \boldsymbol{v}_j$ を $\boldsymbol{x}$ から作る（単位長）。ヘッド h の重みは
$W^h_{ij} = \alpha(s^h_{ij})\, m_h(d_{ij})$、　$s^h_{ij} = \hat{\boldsymbol{q}}^h_i \cdot \hat{\boldsymbol{k}}^h_j$、　$\alpha(s) = \big((1+s)/2\big)^4$
m_h：ヘッドごとの入れ子の窓（距離 d で 1 から 0 へ）。$\sum_j W^h_{ij}\boldsymbol{v}^h_j$（softmax の分母はない）→ TanhNorm・ゲート・W_O でヘッドを足す
→ MLP（残差に足す）→ 各電子の特徴 $\boldsymbol{h}_i$

軌道　**lj_orbitals.py**
$\phi^k_j(\boldsymbol{r}_i, s_i)$
$= \phi^{\text{HF}}_j(\boldsymbol{r}_i, s_i) + \varepsilon\,[W^k_{s_i}\boldsymbol{h}_i]_j$
ϕ^{HF}：① の HF の表、$\varepsilon = 0.05$、k：行列式の番号

出力
行列式 4 枚（$k = 1,\dots,4$）の和 $\times\, e^J$（J：擬スピンの組ごとの Jastrow、カスプつき）
→ $\ln\psi_\theta(R)$（複素）

学習で動く θ：W_{emb}、W_q, W_k, W_v、ゲート、W_O、MLP、$W^k_\pm$、Jastrow の長さなど。　固定：窓 m_h、Trim の幅（$r = 2$）、ϕ^{HF}。　⑦ の **readout3.py** が読むのは遮蔽アテンションの s^h_{ij}、α、m_h。

フォルダの並び　PC と GitHub で同じ。☆ は PC にだけある（GitHub に上げない）

periodicwave/　計算の本体（③④⑤）。いまの設定は **configs/run_lambda_jellium.py**
screening_analysis/　解析。いまの λ-jellium は **lj_hf_branch.py**・**lj2/**・**lj3/**。ほかの 19 本は共有の部品（右）
test_*.py　検算（中）　**archive/**　いまは使っていないもの（2DEG・モアレ・窓の設計、λ-jellium の最初のスキャン）
doc/code_map/　この図。ほかの **doc/** は ☆
☆ **ws_pw_kit/**　投入と回収（②⑥）、HF の表 **lj2_hf/**、回収した結果 **pulled_lj3/**・**pulled_lj2/**、古い結果 **pulled_old/**
☆ **gh_kit/**　GitHub に送る道具（**_push_wada_dev.bat**）　☆ **_to_delete/**　整理で外したもの（消してよいか確かめてから消す）

検算　クラウドか PC の CPU

test_lambda_jellium.py　局所エネルギーを差分の $[H\psi]/\psi$ と比べる（入力 12 成分のネットワークも）、擬スピンのモンテカルロの分布、反転＋半分ずらしの対称性、初期値 match で関数が変わらないこと
test_screenformer.py　反対称性、格子の周期、勾配とラプラシアンが有限
test_gauss_rings.py　ガウスでぼかした輪の窓（いまは使っていない設定）

共有の部品　screening_analysis/ の 19 本

2DEG・モアレの時期に作り、ほかの解析やテストが import するので残したもの
style.py　sampler.py　attn_probe.py　head_contrib.py　head_cancel.py　psi_probe.py　reach.py　moire_analysis.py　moire_summary.py　graph_wind.py　lj_maps.py　bigN_maps.py　fig_channels.py　fig_fig6.py　fig_qk.py　fig_kernel.py　fig_order.py　fig_lambda.py　analyze_all.py